Annexe A : Sujets bonus

Introduction

À ce stade, vous en savez assez pour commencer à utiliser git de façon efficace au travail en apprenant le reste par vous-même. Mais si vous souhaitez en savoir plus, ce TP complémentaire peut vous présenter plusieurs sujets plus avancés.

Contrairement aux TPs précédents, il n'y a pas d'ordre, ni de marche à suivre précise. Prenez les différentes « parties » dans l'ordre que vous souhaitez, et essayez ces nouvelles fonctionnalités sur des dépôts git de votre choix, par exemple ceux que vous avez créés au cours de la formation !

La double vie de `git mv`

Au cours du premier TP, nous avons présenté `git mv` comme une commande qui déplace un fichier sous gestion de version. En réalité, ce n'est pas exactement ce qui se passe.

Lorsqu'on exécute la commande `git mv`, git effectue sous le capot les opérations suivantes :

- Déplacer le fichier au sein du répertoire de travail
- Noter que le fichier a été supprimé de son ancien emplacement
- Noter qu'un fichier a été ajouté à son nouvel emplacement

Un mouvement de fichier est donc traité exactement comme une suppression suivie d'une re-création, et c'est ensuite une optimisation de performance interne à git qui s'assure que cette opération soit implémentée de façon efficace.

C'est équivalent, me direz-vous. En fait, c'est même mieux, parce que cela permet à git de traiter plus efficacement des opérations qui sont *presque* des déplacements (ex : déplacement avec ajout mineur de contenu). Cependant, cela conduira aussi parfois les outils construits autour de git à échouer à identifier un déplacement de fichier lorsque cela leur serait utile.

Identifier l'origine d'un problème

L'outil `git bisect` permet de localiser rapidement l'origine d'un problème dans l'historique du dépôt en utilisant une recherche par dichotomie. Il est d'autant plus efficace que les recommandations données précédemment concernant les commits (peu de changements par commit, et des commits auto-suffisants) ont bien été suivies.

Pour utiliser cet outil, il faut procéder de la façon suivante. D'abord, on active la recherche par dichotomie :

```
$ git bisect start
```

Ensuite, on désigne un commit incorrect avec `git bisect bad [<commit>]`. Si on ne spécifie pas quel commit est mauvais, c'est le commit courant qui sera sélectionné par défaut :

```
$ git bisect bad
```

Puis on désigne un commit dans le passé qui était correct, avec la commande `git bisect good [<commit>]`, qui fonctionne comme `git bisect bad` :

```
$ git bisect good <commit correct>
```

À partir de là, git va commencer à chercher en quel point de l'historique les choses ont mal tourné, via une recherche par dichotomie. À chaque commit où `git bisect` s'arrête, il vous sera demandé de dire si ce commit est correct ou non, avec `git bisect good` ou `git bisect bad`. Et au bout d'un faible nombre d'itérations, git vous indiquera quel commit semble être à l'origine du problème que vous étudiez.

Il est aussi possible d'automatiser complètement `git bisect` en utilisant `git bisect run <commande>`, qui prend en argument un programme capable de vérifier si le commit actif est correct ou non. Consultez la documentation de `git bisect` pour plus d'informations sur cette fonctionnalité.

Chercher une information dans le dépôt

Vous êtes sans doute familier avec la commande `grep`, qui permet de rechercher un texte au sein d'une hiérarchie de fichiers. git fournit une alternative à celle-ci, `git grep`, qui…

1. Tire parti des informations connues de git pour effectuer la recherche beaucoup plus rapidement, ce qui est précieux dans les gros dépôts.
2. Est capable de rechercher des informations non seulement dans le répertoire de travail, mais aussi dans l'historique des commits.
3. Se restreint aux fichiers sous gestion de version.

En contrepartie, cette commande nécessite un temps d'adaptation pour l'habitué de GNU grep, car sa logique est un peu différente. Par exemple, le dialecte d'expression régulière par défaut n'est pas le même.

La manière la plus simple d'utiliser `git grep` est :

```
$ git grep 'Une recherche passionnante'
```

Une option très utile est `-E`, qui active le support des expressions régulières Perl. Le prix à payer étant qu'il y a davantage de caractères spéciaux que l'on doit parfois « échapper » avec `\` dans la chaîne recherchée :

```
$ git grep -E '(ce(ci|la)?|ça)'
```

Guide complet de `reset` et `checkout`

Parmi les commandes git d'usage courant, les plus difficiles à maîtriser sont sans doute `git reset` et `git checkout`. Cela tient au fait qu'elles font un grand nombre de choses différentes selon ce qu'on leur passe en paramètre. Passons donc au crible leurs différentes variantes.

`git checkout <commit>` est une commande qui sert à se promener dans l'historique. Elle bascule `HEAD` sur un commit donné, et met à jour l'index et le répertoire de travail pour qu'ils collent au contenu de ce commit. Il vaut donc mieux l'exécuter avec un répertoire de travail propre, sinon elle échouera avec un message d'erreur.

Cette commande change également de branche active. Si on lui passe une branche en paramètre, ladite branche devient la branche active, au sein de laquelle les nouveaux commits sont enregistrés. Si on passe à `checkout` quelque chose qui n'est pas une branche (une étiquette, un *hash* de commit…), alors il n'y aura plus de branche active. On parlera de « `HEAD` détachée ».

Comme nous l'avons vu au TP 4, cette commande peut aussi créer automatiquement une branche locale liée à une branche distante.

Toujours dans les raccourcis, `git checkout -b <nom>` est une abréviation de `git branch <nom> && git checkout <nom>` : on crée une branche et on bascule dessus.

`git checkout <fichiers>` modifie des fichiers du répertoire de travail pour qu'ils correspondent à leur version dans l'index. Vous pouvez le voir comme l'inverse de la commande `git add <fichiers>`, qui modifie l'index pour qu'il corresponde au contenu du répertoire de travail.

`git checkout <commit> -- <fichiers>` sert à récupérer et mettre dans le répertoire de travail et l'index des versions de fichier issues d'un certain commit.

La branche active et le pointeur `HEAD` ne sont pas affectés par ces deux dernières formes de `git checkout`. Comme `git add`, `git checkout` supporte aussi une option `--patch` qui permet de récupérer des changements depuis la source choisie à une granularité plus fine que le fichier.

Voilà pour `git checkout`. Maintenant, passons à `git reset`.

`git reset <commit>` est une commande qui sert à déplacer la branche active (et `HEAD`) sur un autre commit. Elle se distingue en cela de `checkout`, qui change `HEAD` mais ne déplace pas la branche active.

Avec cette commande, il est possible de rendre des commits inaccessibles. Assurez-vous donc d'avoir bien compris comment utiliser les `reflogs` avant d'utiliser cette variante de `git reset`, afin de pouvoir facilement ramener votre branche au point précédent en cas de manipulation incorrecte.

Cette commande possède trois variantes, correspondant à différents niveaux d'agressivité vis-à-vis de l'index et du répertoire de travail :

- `git reset --soft <commit>` déplace uniquement la branche active, en laissant l'index et le répertoire de travail tels quels. Comme on l'a vu précédemment, on peut notamment s'en servir pour annuler un appel prématuré à `git commit`.
- `git reset [--mixed] <commit>`, qui est le mode par défaut, met également à jour l'index pour qu'il corresponde au contenu du nouveau `HEAD`. Il n'y a donc plus de « changements prêts à être commités » dans `git status` après exécution de cette commande.
- `git reset --hard <commit>` propage également les versions de fichiers de `HEAD` au sein du répertoire de travail. **Cette opération peut donc perdre des données présentes au sein de ce dernier de façon irréversible**, et il faut l'utiliser avec précaution._ Mais c'est aussi l'un des moyens les plus simples de « ramener une branche en arrière » après des commits incorrects.

`git reset [<commit>] -- <fichiers>` récupère la version des fichiers choisis du commit choisi (`HEAD` par défaut) et l'extrait dans l'index. L'utilisation la plus courante de cette variante est d'annuler l'effet de `git add` en ramenant certains fichiers de l'index au point où ils étaient dans `HEAD`.

Il existe un parallèle important entre cette dernière commande et la variante de `git checkout` qui prend des noms de fichiers en paramètre. La seule différence entre les deux est qu'on opère sur l'index au lieu d'opérer sur le répertoire de travail. Ici aussi, l'option `--patch` permet de contrôler plus finement quels changements on effectue au sein de l'index.

Savoir d'où vient une partie d'un fichier

Parfois, il arrive que l'on souhaite savoir qui est l'auteur d'une partie du texte présent au sein du dépôt. Comme c'est rarement pour faire des éloges à cette personne, la commande pour le faire s'appelle `git blame`.

Quand on exécute `git blame <fichier>`, git affiche une version annotée du fichier en question, comprenant le *hash* du dernier commit où ce fichier a été modifié, la date de modification, et le nom de l'auteur de cette dernière.

On peut ensuite afficher la nature exacte des modifications qui ont été effectuées, par exemple avec `git show <commit>` ou en faisant un *checkout* sur le commit correspondant. Cela permettra de vérifier que la personne est bien l'auteur initial de la partie du fichier concerné. En effet, le moindre reformatage d'un document marquera du point de vue de git la personne qui a effectué l'opération comme l'auteur de la section reformatée...

Édition d'historique avec le rebase interactif

Pour terminer en beauté ce petit tour des fonctionnalités avancées de git, mentionnons un outil aussi dangereux que puissant, qui permet de modifier l'historique des commits d'une façon beaucoup plus profonde que tout ce que nous avons vu jusqu'à présent : `git rebase --interactive <commit>`.

Pour rappel, un *rebase* est une opération consistant à rejouer les commits d'une branche par-dessus un nouveau commit parent. Lorsque nous avons introduit cette fonction, nous l'avons présentée comme un moyen de « mettre à jour » une branche quand sa branche parente a reçu de nouveaux commits.

Mais lorsqu'on ajoute y l'option `--interactive`, ou son raccourci `-i`, git nous permet en plus d'intervenir sur le processus par lequel les commits de la branche sont rejoués, et d'agir dessus de différentes manières. On peut ainsi...

- Modifier la description de certains commits de la branche
- Modifier le contenu de ces commits (nouveaux fichiers, suppressions...)
- Fusionner plusieurs commits en un seul
- Supprimer purement et simplement des commits de l'historique

Il va de soi que c'est une technique relativement dangereuse, dont il faut user avec précaution. En plus des risques habituels du *rebase*, la possibilité de modifier le contenu de commits et de supprimer ces derniers peut aussi conduire à éliminer accidentellement des changements d'une branche.

Ces pertes de données ne sont qu'apparentes, et il est possible d'annuler un *rebase* interactif, soit en créant préalablement un tag de sauvegarde à l'ancien emplacement de la branche et en faisant un `git reset --hard` dessus en cas de problème, soit à la force des poings en retrouvant le point du *reflog* où se trouvait la branche avant l'exécution pour effectuer un `reset` dessus.

Mais il n'en reste pas moins que réparer les dégâts d'un *rebase* interactif incorrect demande une bonne maîtrise de git, et qu'il est en ce sens quelque peu regrettable que certains projets logiciels forcent l'usage de ces techniques relativement avancées chez leurs contributeurs.

Git LFS (*Large File Storage*)

Dans sa conception, git est surtout optimisé pour stocker des documents textuels. Son mécanisme de stockage est optimisé pour des fichiers texte bruts dont peu de lignes changent à chaque *commit*, et des outils comme `git diff` et `git merge` ne fonctionnent bien que dans ce contexte.

Mais dans la vraie vie, on n'utilise pas que du texte brut. Un dépôt git peut tout aussi bien se retrouver lié à des fichiers binaires tels que des documents texte enrichis, des images, des archives compressées, ou même des images de machines virtuelles.

Pour ce genre d'usage, git va assez mal se comporter : le calcul de différences ne va plus marcher, ce qui signifie que...

- Toute fusion de branches où les deux branches source ont touché au fichier binaire sera considérée comme un conflit, ce qui limite les possibilités de collaboration.
- À chaque modification d'un fichier binaire, on en stockera une nouvelle copie complète, ce qui peut prendre beaucoup de place sur le disque et ralentir les échanges réseau.

Pour limiter l'impact de ce second problème, les équipes de la forge GitHub ont mis au point une extension de git appelée Git *Large File Storage* (généralement abrévié en Git LFS), qui permet de s'assurer que git ne stocke plus l'ensemble des versions des fichiers binaires, mais seulement celles dont on a eu besoin à un moment donné. Lorsqu'une nouvelle version est requise, elle est téléchargée depuis le serveur.

Les conséquences sont les suivantes :

- Côté serveur, peu de changements, on stocke toujours toutes les versions du fichier et il faut donc prévoir de l'espace de stockage en conséquence.
- Côté client, on ne possède plus une copie complètement indépendante du dépôt git, le `checkout` d'anciens commits peut nécessiter une connexion internet.
- Il faut que le serveur (la forge) soit compatible avec LFS, et que chaque utilisateur du dépôt ait installé l'extension git LFS.

Insistons un peu sur ce dernier point, car il est important. Si tous les utilisateurs du dépôt ne sont pas équipés de l'extension Git LFS, alors il est très facile de se retrouver dans une situation où les gros fichiers sont remplacés par des simples identifiants textuels (utilisés par Git LFS en interne). Il est donc important que la décision d'utiliser Git LFS et les prérequis associés soient communiqués clairement à l'ensemble de l'équipe.

Vous trouverez plus d'information et des instructions d'installation à l'adresse https://git-lfs.github.com/ .